GEORISK MONITOR

Sistema de Alerta de Riscos Geográficos e Climáticos

Zona Leste de São Paulo - Tatuapé

Documento de Especificação de Software

Integrantes

- Éric da Silva Barros - 44726511
- Evandro Benati - 45891907
- Hellen Ariane Bastos de Oliveira - 45771189
- Isabela Araujo de Santana - 44706448
- Larissa Santos Ferreira - 45648379
- Maria Eduarda Lemos Pelegrin Machado - 42709407
- Pedro Henrique Ferreira Lima - 45795312
- Zulma Vicente Cespedes - 4371072

SUMARIO

1. Missão, Visão e Valores

1.1 Missão

1.2 Visão

1.3 Valores

2. Objetivo do Projeto

2.1 O Problema Real

2.2 A Solução

2.3 O Que o Sistema Oferece

3. Escopo da Entrega

3.1 O Que o Sistema Faz

3.2 O Que Não Está Incluído

4. Termos e Definições Importantes

5. Como o Sistema Funciona

5.1 A Experiencia do Usuario

5.2 Como os Alertas Funcionam

5.3 O Indice de Risco

5.4 Como as Páginas Sao Organizadas

6. Requisitos do Sistema

6.1 O Que o Sistema Deve Fazer

6.2 Como o Sistema Deve Se Comportar

7. Arquitetura do Sistema

7.1 Visão Geral

7.2 O Frontend — A Parte Visual

7.3 O Backend — O Servidor Java

7.3.1 Os Controllers — Quem Atende as Requisições

7.3.2 As Entities — Como os Dados Sao Estruturados

7.3.3 A Segurança — JWT e BCrypt

7.4 Integração com a API Open-Meteo

8. Banco de Dados

8.1 Por que MySQL

8.2 Scripts SQL Executados

8.2.1 Criando e selecionando o banco de dados

8.2.2 Criando a tabela de usuários

8.2.3 Verificando o banco e as tabelas

8.2.4 Inserindo e consultando um usuario de teste

8.2.5 Criando a tabela de alertas

8.2.6 Adicionando a coluna de vínculo com o usuário

8.2.7 Verificando a estrutura da tabela de alertas

8.3 Diagrama Entidade-Relacionamento (DER)

8.4 ERP — Entidade, Relacionamento e Persistência

8.5 Configuração da Conexão com o Banco

9. Fluxos Principais do Sistema

9.1 Fluxo de Cadastro

9.2 Fluxo de Login

9.3 Fluxo de Registro de Alerta

9.4 Fluxo de Atualização Automática

10. Riscos e Limitações

10.1 Riscos Conhecidos

10.2 Limitações da Versão Atual

11. Dependências Externas

12. Referencias

1. Missão, Visão e Valores

O GeoRisk Monitor surgiu de uma necessidade real: moradores da Zona Leste de São Paulo, especialmente do bairro do Tatuapé, convivem todo ano com o medo das chuvas fortes. Ruas que viram rios, carros submersos, famílias ilhadas. A falta de informação rápida e confiável sobre o que está acontecendo ao redor e um problema que afeta a vida dessas pessoas de forma direta. O sistema foi criado para resolver isso de uma forma simples: mostrar, em tempo real, onde os riscos estão acontecendo e avisar quando o perigo está chegando.

1.1 Missão

A missão do GeoRisk Monitor e entregar, gratuitamente e em tempo real, uma ferramenta de monitoramento que avisa moradores e gestores públicos quando existe risco de alagamento ou deslizamento na região do Tatuapé. Não é um sistema complexo feito para especialistas. E feito para qualquer pessoa com um celular na mão que precisa saber se pode sair de casa com segurança ou se deve ligar para a defesa civil.

1.2 Visão

O objetivo e que o GeoRisk Monitor se torne, até 2027, a principal referencia em prevenção de desastres climáticos na Zona Leste de São Paulo. Com o tempo, a ideia e expandir a cobertura para outros bairros vulneráveis da cidade e servir de modelo para que outras cidades brasileiras com problemas parecidos possam adotar a mesma solução.

1.3 Valores

Toda decisão tomada durante o desenvolvimento do sistema foi guiada por um conjunto de valores que a equipe levou a sério desde o início.

Transparência. O sistema não esconde nada do usuário. Cada dado climático exibido vem de fontes verificáveis. Cada alerta no mapa mostra quem registrou e quando. Não existe informação manipulada ou omitida.

Acessibilidade. O sistema foi pensado para funcionar em qualquer celular básico com acesso à internet, sem precisar instalar nada. Basta abrir o navegador. Isso e fundamental para alcançar os moradores que mais precisam da ferramenta.

Confiabilidade. Os dados climáticos vêm da API Open-Meteco, uma fonte meteorológica internacional reconhecida. Os alertas registrados pelos usuários são salvos em banco de dados com segurança, sem risco de perda de informação critica.

Privacidade. O sistema coleta apenas o mínimo necessário: nome, e-mail, telefone e localização aproximada. As senhas nunca são salvas em texto aberto — passam sempre por um processo de criptografia chamado BCrypt. Nada e compartilhado com terceiros, em conformidade com a Lei Geral de Proteção de Dados (LGPD).

Compromisso com o bem público. O projeto não tem fins comerciais. Foi desenvolvido como trabalho acadêmico com vocação publica, podendo ser disponibilizado a prefeituras e órgãos de defesa civil para adoção e expansão sem custo.

2. Objetivo do Projeto

2.1 O Problema Real

Quem mora no Tatuapé ou na Zona Leste de São Paulo sabe o que acontece quando chove forte. O rio Tiete transborda, as ruas ficam intransitáveis, bueiros não dão conta da água e famílias inteiras ficam ilhadas. Segundo o Centro de Gerenciamento de Emergências Climáticas da prefeitura de São Paulo (CGE-SP).

O problema não e só a chuva em si. E a falta de informação no momento certo. Quando o morador percebe que a rua está alagando, muitas vezes já e tarde demais para sair com o carro. Os avisos oficiais chegam por rádio ou televisão com atraso, ou por grupos de aplicativos de mensagem não oficiais, sem localização precisa. A defesa civil, por sua vez, não tem como saber em tempo real quais pontos estão críticos para priorizar o envio de equipes.

Esse cenário de resposta lenta e reativa e exatamente o que o GeoRisk Monitor veio resolver.

2.2 A Solução

O GeoRisk Monitor e um site que funciona no navegador do celular ou computador. Assim que o usuário abre, ele mostra um mapa da região do Tatuapé com todas as informações climáticas atuais: temperatura, velocidade do vento, quantidade de chuva por hora, umidade do ar. Com base nesses dados, o sistema calcula automaticamente um índice de risco — de 0 a 100 — que indica se a situação está tranquila, moderada ou perigosa.

Mas além dos dados automáticos, qualquer morador cadastrado pode clicar no mapa e registrar um alerta no ponto exato onde viu algo preocupante: uma rua alagando, uma enxurrada, uma área com risco. Esse alerta aparece imediatamente para todos os outros usuários conectados. E como um aplicativo colaborativo de alertas, mas focado em riscos climáticos e alagamentos.

O acende do sistema, ou seja, a parte que roda no servidor e processa todas as informações, foi desenvolvido em Java com Spring Boot. O fronteando, que é a parte visual que o usuário vê no navegador, foi feito em HTML, CSS e JavaScript puro — sem frameworks como React ou Vue. Tudo em arquivos .html simples com o CSS e JS embutidos dentro do próprio arquivo. A sincronização dos alertas em tempo real entre diferentes usuários e feita pelo Firebase Firestore, um serviço do Google.

2.3 O Que o Sistema Oferece

Com o GeoRisk Monitor funcionando, os moradores ganham acesso rápido a situação climática da sua região, podendo ver no mapa onde os riscos estão ocorrendo e receber notificações quando o índice de risco fica alto. A defesa civil e os gestores públicos passam a ter dados aeroespaciais em tempo real que podem ajudar a direcionar equipes para os pontos mais críticos com mais agilidade. E a comunidade, no geral, deixa de depender de boatos e informações atrasadas para tomar decisões de segurança.

3. Escopo da Entrega

3.1 O Que o Sistema Faz

O GeoRisk Monitor entrega um conjunto de funcionalidades que cobrem desde o cadastro do usuário até o monitoramento em tempo real. Veja o que está incluído:

- Qualquer pessoa pode criar uma conta informando nome, sobrenome, e-mail, telefone e senha. A senha precisa ter pelo menos 6 caracteres e é armazenada de forma criptografada.
- Após criar a conta, o usuário faz login e recebe um token de acesso com validade de 24 horas, que garante que apenas ele possa registrar ou excluir os próprios alertas.
- O sistema abre um mapa interativo centralizado no Tatuapé, com fundo escuro e marcadores coloridos que indicam os alertas registrados.
- O GPS do celular ou computador do usuário e detectado automaticamente, colocando um marcador azul na posição dele no mapa.
- A cada 5 minutos, o sistema consulta automaticamente os dados climáticos atuais da API Open-Meteo: temperatura, sensação térmica, velocidade do vento, precipitação por hora, umidade e código meteorológico internacional WMO.
- Com base nesses dados, um índice de risco e calculado e exibido no painel lateral do mapa, com uma barra colorida que vai do verde (baixo risco) até o vermelho (risco alto).
- O usuário pode clicar em qualquer ponto dentro da área monitorada — um círculo de 1.700 metros ao redor do Tatuapé — e registrar um alerta, informando o tipo e um comentário opcional.

- Quando as condições climáticas ficam críticas, o sistema gera alertas automáticos no mapa, sem precisar que nenhum usuário clique em nada.
- Todos os alertas aparecem em tempo real para todos os usuários conectados ao mesmo tempo.
- Os alertas podem ser filtrados por tipo no painel lateral: todos, alagamento, chuva forte ou automáticos.
- O usuário pode excluir os próprios alertas. O sistema verifica no servidor se realmente e o dono do alerta antes de permitir a exclusão.
- Quando o índice de risco passa de 70, um banner vermelho pisco na tela e o sistema pode enviar uma notificação pelo próprio navegador, mesmo com o site minimizado.
- Uma previsão do tempo para os próximos 7 dias e exibida no painel lateral, com temperatura máxima, mínima e chance de chuva para cada dia.

3.2 O Que Não Está Incluído

Para deixar claro o que foi entregue nesta versão, e importante listar também o que ficou fora do escopo por ora:

- Não existe aplicativo para Android ou iOS. O sistema funciona apenas pelo navegador do celular ou computador.
- Não há integração direta com sistemas da prefeitura ou da defesa civil. Isso está previsto para versões futuras.
- O sistema funciona apenas em português brasileiro.
- Não existe histórico exportável de dados climáticos. O sistema mostra apenas as informações do momento atual.
- Não é possível gerar relatórios em PDF ou Excel.

- Login com Google, Facebook ou outras redes sociais não está disponível nesta versão.

4. Termos e Definições Importantes

Este documento usa alguns termos técnicos que podem não ser familiares para todos os leitores. A seguir, cada um deles e explicado de forma simples.

API (Application Programming Interface). E uma forma de dois sistemas conversarem entre si. Quando o GeoRisk busca os dados do tempo, ele manda uma pergunta para a API Open-Meteo e recebe as respostas com temperatura, chuva e vento. E como ligar para um serviço de informações e pedir os dados que você precisa.

JWT (JSON Web Token). E uma espécie de crachá digital que o sistema entrega ao usuário quando ele faz login. Esse crachá fica salvo no navegador e é enviado junto com cada ação que o usuário faz, para provar que ele está autorizado. Tem validade de 24 horas.

BCrypt. E o método de criptografia usados para proteger as senhas. Quando você cadastra uma senha, o sistema a transforma em uma sequência embaralhada de caracteres antes de salvar. Mesmo que alguém invada o banco de dados, não consegue descobrir a senha original a partir dessa sequência.

Código WMO. E um número padronizado pela Organização Meteorológica Mundial para descrever as condições do tempo. Por exemplo, o código 0 significa céu limpo, o código 65 significa chuva intensa e o código 95 significa tempestade. O sistema usa esses códigos para identificar automaticamente quando a situação climática e perigosa.

Índice de Risco. E um número calculado pelo sistema que vai de 0 a 100, indicando o nível de perigo climático no momento. Abaixo de 30 e considerado baixo, entre 30 e 59 e moderado e acima de 60 e alto. O índice leva em conta a velocidade do vento, a quantidade de chuva por hora e o código WMO da condição atual.

Area Monitorada. E a região coberta pelo sistema: um círculo de 1.700 metros de raio com centro no Tatuapé, nas coordenadas -23,5405 de latitude e -46,5760 de longitude. Alertas registrados fora dessa área são bloqueados automaticamente.

Firebase Firestore. E um banco de dados em nuvem do Google que permite sincronização em tempo real. Quando um usuário registra um alerta, o Firestore avisa todos os outros usuários conectados na mesma hora, sem precisar atualizar a página.

Alerta Manual. E um alerta criado pelo próprio usuário, quando ele identifica uma situação de risco e clica no mapa para registra-la.

Alerta Automático. E um alerta criado pelo próprio sistema, sem interação do usuário, quando os dados climáticos indicam condições perigosas — como chuva forte acima de 5 mm/hora, vento acima de 40 km/h ou código WMO de tempestade.

5. Como o Sistema Funciona

5.1 A Experiencia do Usuário

A jornada começa na página inicial do GeoRisk Monitor. O usuário vê uma tela escura com o nome do sistema e duas opções: criar uma conta ou entrar com uma conta já existente. O visual foi pensado para ser moderno e funcional, com animações suaves que deixam claro que o sistema está ativo e funcionando.

Depois de criar a conta e fazer login, o usuário e levado direto para a tela principal: o mapa. Nesse momento, o navegador pede permissão para acessar a localização do dispositivo. Se o usuário aceitar, o mapa se centraliza na posição dele e começa a mostrar as informações climáticas da região. Se recusar, o mapa centraliza no ponto de referência do Tatuapé mesmo assim.

No lado esquerdo da tela, um painel mostra as condições climáticas atuais — temperatura, sensação térmica, quantidade de chuva, umidade do ar e velocidade do vento — junto com o índice de risco calculado e a previsão para os próximos 7 dias. No centro, o mapa interativo com todos os alertas. No topo, um contador regressivo mostra quando será a próxima atualização automática dos dados.

5.2 Como os Alertas Funcionam

Para registrar um alerta, o usuário clica em qualquer ponto dentro do círculo que delimita a área monitorada. Uma janela aparece pedindo que ele escolha o tipo de alerta — alagamento ou chuva forte — e escreva um comentário opcional descrevendo o que está vendo. Quando confirma, o alerta aparece no mapa com um marcador colorido: vermelho para alagamento e azul para chuva forte.

Esse alerta e visível para todos os outros usuários conectados na mesma hora. Não é necessário atualizar a página. O Firebase Firestore cuida disso automaticamente, atualizando o mapa de todos em tempo real.

Os alertas automáticos funcionam de forma diferente: o sistema verifica, a cada 5 minutos, se os dados climáticos estão acima dos limites de risco. Se estiverem, ele cria marcadores amarelos no mapa automaticamente, sem que ninguém precise fazer nada. Esses marcadores pulsam para chamar atenção.

5.3 O Índice de Risco

O índice de risco e calculado toda vez que os dados climáticos são atualizados. O cálculo considera três fatores: a velocidade do vento contribui com até 40 pontos, a quantidade de chuva por hora contribui com até 40 pontos, e o código WMO da condição climática adiciona até 20 pontos extras quando a situação e de chuva intensa, tempestade ou granizo.

Se o resultado for menor que 30, a barra fica verde e o sistema indica situação de baixo risco. Entre 30 e 59, fica amarela com risco moderado. Acima de 60, fica vermelha com risco alto. Quando passa de 70, o sistema ativa um banner de alerta vermelho na parte superior da tela e pode enviar uma notificação pelo navegador, avisando o usuário mesmo que ele esteja com o site minimizado.

5.4 Como as Páginas São Organizadas

O sistema tem quatro páginas no total. A página georisk.html e a entrada principal, onde ficam os formulários de login e cadastro em uma interface única. A página georisk_mapa.html e o coração do sistema, onde o mapa, os alertas e as informações climáticas são exibidas. As páginas cadastro.html e login.html são versões alternativas mais simples dos mesmos formulários.

Quando o usuário tenta acessar o mapa sem estar logado, o sistema percebe isso no momento em que a pagina carrega e redireciona automaticamente para a tela de login. Isso acontece graças ao Firebase Authentication, que verifica o estado da sessão assim que a pagina abre.

6. Requisitos do Sistema

6.1 O Que o Sistema Deve Fazer

Os requisitos funcionais descrevem tudo que o sistema precisa ser capaz de realizar para cumprir seu propósito.

RF01 - Registrar alertas no mapa. O usuário autenticado deve poder clicar em qualquer ponto dentro da área monitorada e registrar um alerta com tipo e comentário. O alerta precisa aparecer imediatamente no mapa de todos os outros usuários conectados.

RF02 - Gerar alertas automáticos. O sistema deve identificar sozinho quando as condições climáticas são perigosas e criar alertas automáticos no mapa, sem precisar de ação do usuário.

RF03 - Calcular e exibir o índice de risco. O sistema deve calcular um número de 0 a 100 representando o nível de perigo climático atual e exibi-lo visualmente no painel lateral com uma barra colorida.

RF04 - Gerenciar contas de usuário. O sistema deve permitir cadastro, login, recuperação de senha por e-mail e logout. O usuário deve conseguir ver o próprio nome exibido no cabeçalho quando estiver logado.

RF05 - Mostrar previsão de 7 dias. O painel lateral deve exibir as condições esperadas para os próximos 7 dias, com temperatura máxima, mínima e chance de chuva para cada dia.

RF06 - Filtrar e excluir alertas. O usuário deve poder filtrar os alertas exibidos no mapa por tipo. Alertas criados pelo próprio usuário devem poder ser excluídos, e o sistema deve confirmar no servidor que só o dono pode fazer isso.

6.2 Como o Sistema Deve Se Comportar

Os requisitos não funcionais descrevem a qualidade e o comportamento esperado do sistema, além das funcionalidades em si.

Velocidade. O mapa deve carregar e mostrar os alertas existentes em menos de 3 segundos em uma conexão comum de internet.

Disponibilidade. O servidor deve estar funcionando 24 horas por dia, 7 dias por semana. O tempo de resposta para 95% das requisições deve ser inferior a 2 segundos em condições normais de uso.

Segurança das senhas. Nenhuma senha pode ser guardada em texto puro no banco de dados. Toda senha passa pelo processo de criptografia BCrypt antes de ser salva.

Proteção das rotas. Qualquer ação que exija autenticação deve retornar erro se o token JWT estiver ausente, invalido ou expirado. Isso inclui registrar alertas, listar os próprios alertas e excluir alertas.

Facilidade de uso. A interface deve ser simples o suficiente para que um morador sem experiencia técnica consiga registrar um alerta sem precisar de ajuda ou treinamento.

Conformidade com a LGPD. O sistema deve coletar apenas os dados necessários e permitir que o usuário solicite a exclusão dos próprios dados a qualquer momento.

7. Arquitetura do Sistema

7.1 Visão Geral

O GeoRisk Monitor e dividido em três partes bem separadas: o fronteando, que é tudo que o usuário vê na tela; o acende, que é o servidor responsável por processar as regras do sistema e guardar as informações; e o banco de dados, onde os dados ficam armazenados de forma permanente. Essa separação e uma boa pratica de desenvolvimento porque permite que cada parte evolua de forma independente sem quebrar as outras.

Além dessas três partes principais, o sistema também usa dois serviços externos: a API Open-Meteo, que fornece os dados climáticos, e o Firebase do Google, que cuida da autenticação dos usuários e da sincronização dos alertas em tempo real.

7.2 O Frontend — A Parte Visual

O frontend foi construído com tecnologias básicas da web: HTML, CSS e JavaScript puro — sem frameworks como React ou Vue. Tudo em arquivos .html simples com o CSS e JS embutidos dentro do próprio arquivo, o que torna o código mais leve e fácil de entender.

O mapa interativo e gerado pela biblioteca Leaflet.js, uma das mais usadas no mundo para criar mapas em aplicações web. Os tiles do mapa — ou seja, as imagens que compõem o visual do mapa — vem do CartoDB Dark Matter, que oferece um fundo escuro e de alta legibilidade, ideal para destacar os marcadores coloridos de alerta.

O fundo animado com partículas que se movem e se conectam por linhas foi implementado usando o Canvas 2D, uma API nativa dos navegadores modernos. Nenhuma biblioteca externa foi usada para esse efeito — todo o código foi escrito manualmente.

A comunicação com o servidor Java e feita por chamadas de requisição HTTP assíncronas. O token JWT e enviado no cabeçalho de cada requisição que exige autenticação, no formato "Authorization: Bearer [token]".

7.3 O Backend — O Servidor Java

O backend e uma API REST desenvolvida em Java 21 com o framework Spring Boot na versão 3.3.5. Uma API REST e um servidor que recebe pedidos pelo protocolo HTTP e responde com dados em formato JSON, o mesmo formato que o JavaScript do frontend usa para trabalhar com informações.

O projeto segue a arquitetura padrão do Spring Boot, organizada em camadas com responsabilidades bem definidas. A camada de controllers recebe e responde as requisições. A camada de entities define como os dados são estruturados. A camada de repositores cuida do acesso ao banco de dados. E a camada de security cuida da autenticação e autorização.

7.3.1 Os Controllers — Quem Atende as Requisições

O AuthController e responsável por tudo que envolve cadastro e login. Quando um novo usuário se cadastra, esse controller recebe os dados, verifica se o e-mail já existe, criptografa a senha com BCrypt e salva o usuário no banco. Em seguida, gera um token JWT e devolve para o frontend. No login, o processo e semelhante: verifica o e-mail, compara a senha com o hash salvo e, se estiver tudo certo, gera e devolve o token.

O AlertaController gerencia tudo que envolve os alertas. Ele tem um endpoint público para listar todos os alertas, que o frontend usa para carregar os marcadores no mapa. Tem um endpoint protegido para criar novos alertas, que exige token JWT valido. E tem um endpoint para excluir alertas, que além do token verifica se quem está pedindo a exclusão e de fato o dono do alerta — caso contrário, devolve erro de acesso negado.

O UsuarioController tem apenas um endpoint, que devolve as informações do perfil do usuário logado. O HelloController e apenas uma rota de verificação

que retorna "API funcionando" quando chamada, servindo para checar se o servidor está no ar.

7.3.2 As Entities — Como os Dados São Estruturados

A entity usuário representa um usuário cadastrado no sistema, com os campos id, nome, e-mail, telefone e senha. O campo e-mail tem uma restrição de unicidade, ou seja, dois usuários não podem ter o mesmo e-mail.

A entity alerta representa um registro de ocorrência no mapa, com os campos id, tipo, comentário, latitude, longitude, automático e data Hora. O campo automático indica se o alerta foi criado pelo sistema ou pelo usuário. Ha também um relacionamento com a entity usuário, indicando quem criou o alerta — esse campo pode estar vazio para alertas automáticos.

A entity Ponto Critico representa localizações pré-cadastrados com histórico de risco de alagamento, com os campos id, nome, latitude, longitude e risco Alagamento. Essa entidade está preparada para uso em versões futuras do sistema.

7.3.3 A Segurança — JWT e BCrypt

A segurança do backend e implementada em três peças que trabalham juntas. A classe JwtUtil gera e valida os tokens JWT. Quando o usuário faz login, ela cria um token assinado digitalmente com uma chave secreta definida no arquivo de configuração. Esse token carrega o e-mail e o ID do usuário dentro dele, de forma que o servidor possa saber quem está fazendo a requisição sem precisar consultar o banco de dados a cada vez.

A classe JwtFilter e executada automaticamente para cada requisição que chega ao servidor. Ela lê o cabeçalho da requisição, extrai o token, verifica se a assinatura e valida e se não está expirado. Se estiver tudo certo, registra a identidade do usuário no sistema de segurança do Spring para que os controllers possam acessa-la. Se o token for invalido ou estiver ausente em uma rota protegida, a requisição e bloqueada com erro 401.

A classe SecurityConfig define quais rotas são públicas e quais exigem autenticação. As rotas de cadastro e login são públicas para que qualquer pessoa possa acessa-las. A rota que lista todos os alertas também e publica, já que o mapa pode ser visualizado sem login. Todas as outras rotas exigem token JWT valido.

7.4 Integração com a API Open-Meteo

A consulta dos dados climáticos e feita diretamente no frontend, por meio de uma chamada a API Open-Meteo. Essa API e gratuita, não exige cadastro e oferece dados meteorológicos com cobertura global baseados em modelos de previsão de alta resolução.

A chamada e construída com as coordenadas reais do usuário, obtidas pelo GPS do dispositivo. Isso significa que os dados exibidos no sistema não são genéricos para São Paulo, mas sim específicos para a localização exata do usuário naquele momento. A resposta inclui a condição atual, os dados horários para as próximas 6 horas e a previsão diária para 7 dias — tudo em uma única chamada.

8. Banco de Dados

8.1 Por que MySQL

O banco de dados escolhido para o projeto foi o MySQL na versão 8. Trata-se de um sistema gerenciador de banco de dados relacional, gratuito e amplamente utilizado no mercado. A escolha foi feita por três razoes principais: ele se integra facilmente com o Java por meio do Spring Data JPA, e estável para projetos de médio porte como este, e pode ser administrado visualmente pelo MySQL Workbench, o que facilita bastante durante o desenvolvimento e os testes.

O Hibernante, que faz parte do Spring Data JPA, e responsável por traduzir automaticamente as classes Java para tabelas no banco. Gracia a configuração ddl-auto=update no arquivo de propriedades, toda vez que o servidor Java e iniciado, o Hibernate verifica se as tabelas já existem e, se não existirem, as crias automaticamente. Isso significa que não e preciso rodar os scripts de criação toda vez que o ambiente for configurado do zero, o Hibernate faz isso sozinho.

8.2 Scripts SQL Executados

Os scripts a seguir foram executados no MySQL Workbench na ordem em que aparecem. Cada bloco e acompanhado de uma explicação do que ele faz e por que foi escrito dessa forma.

8.2.1 Criando e selecionando o banco de dados

```
CREATE DATABASE climas_rotas;
USE climas_rotas;
```

O primeiro comando cria um banco de dados novo com o nome climas_rotas. O MySQL 8 ja usa por padrão o charset utf8mb4, que suporta acentos, cedilha e todos os caracteres especiais do português. O segundo comando, USE, diz

ao MySQL que tudo que for executado a partir daqui pertence a esse banco. Sem ele, o MySQL não saberia em qual banco criar as tabelas.

8.2.2 Criando a tabela de usuários

```
CREATE TABLE usuarios (
    id       INT  AUTO_INCREMENT PRIMARY KEY,
    nome     VARCHAR(50),
    email    VARCHAR(100) UNIQUE,
    telefone VARCHAR(20),
    senha    VARCHAR(20)
);
```

Esta tabela guarda os dados de cada pessoa que se cadastra no sistema. O campo id e o identificador unico de cada usuário e é gerado automaticamente pelo MySQL toda vez que um novo registro e inserido — nunca se repete. O campo email tem a restrição UNIQUE, que impede que duas pessoas se cadastrem com o mesmo endereço de e-mail. Se alguém tentar, o banco recusa a operação automaticamente.

O campo senha usa VARCHAR(20) nesta versão inicial. Vale destacar que, na integração com o backend Java, o Spring Security gera um hash BCrypt da senha antes de salva-la — o que transforma a senha em uma sequência embaralhada de caracteres. Mesmo que alguém consiga acessar o banco de dados, não consegue descobrir a senha original a partir desse hash.

8.2.3 Verificando o banco e as tabelas

```
SHOW DATABASES;
SHOW TABLES;
```

O comando SHOW DATABASES lista todos os bancos de dados existentes no servidor MySQL naquele momento. Foi usado para confirmar que o banco climas_rotas foi criado corretamente. O SHOW TABLES lista todas as tabelas

dentro do banco que está selecionado. Foi usado para verificar se a tabela usuarios apareceu após o CREATE TABLE.

8.2.4 Inserindo e consultando um usuário de teste

```
INSERT INTO usuarios (nome, email, telefone, senha)
VALUES ('Teste', 'teste@email.com', '11999999999', '123456');

SELECT * FROM usuarios;
```

O INSERT foi usado para testar se a tabela estava funcionando corretamente, inserindo um usuário fictício com dados simples. O SELECT * FROM usuarios retorna todos os registros da tabela, permitindo visualizar o que foi salvo. Essa etapa e importante para confirmar que a estrutura da tabela está correta antes de conectar o banco com o sistema Java.

Importante: nesse teste, a senha foi inserida em texto puro. Na aplicação real, isso nunca acontece — a senha sempre passa pelo processo de criptografia BCrypt no backend antes de ser enviada para o banco.

8.2.5 Criando a tabela de alertas

```
CREATE TABLE alertas (
    id         INT  AUTO_INCREMENT PRIMARY KEY,
    tipo       VARCHAR(50),
    comentario TEXT,
    latitude   DOUBLE,
    longitude  DOUBLE,
    data       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Esta tabela guarda cada alerta registrado no mapa pelos usuarios ou gerado automaticamente pelo sistema. O campo tipo indica a categoria do alerta, que pode ser "alagamento" ou "chuva". O campo comentario usa o tipo TEXT, que aceita textos de qualquer tamanho — ao contrário do VARCHAR, que tem limite fixo de caracteres.

Os campos latitude e longitude são do tipo DOUBLE, um formato numérico com muitas casas decimais. Isso e necessário porque coordenadas geográficas precisam de alta precisão para localizar um ponto exato no mapa. O campo data e preenchido automaticamente pelo MySQL com a data e hora do momento exato em que o alerta foi criado, graças ao DEFAULT CURRENT_TIMESTAMP. Não e preciso enviar esse valor pelo sistema — o banco cuida disso sozinho.

8.2.6 Adicionando a coluna de vínculo com o usuário

```
ALTER TABLE alertas ADD COLUMN usuario_id INT;
```

Esse comando foi executado depois da criação inicial da tabela alertas para adicionar a coluna usuario_id. Ela serve para registrar qual usuário criou cada alerta. O ALTER TABLE permite modificar uma tabela ja existente sem perder os dados que ja estão nela — e um recurso importante em projetos que evoluem ao longo do tempo.

A coluna usuario_id pode ficar vazia, ou seja, aceita valor nulo. Isso e intencional: os alertas gerados automaticamente pelo sistema com base nos dados climáticos não tem um usuário como autor, então não precisam preencher esse campo.

8.2.7 Verificando a estrutura da tabela de alertas

```
DESCRIBE alertas;
```

O comando DESCRIBE exibe a estrutura completa de uma tabela: os nomes das colunas, os tipos de dados de cada uma, se aceitam valores nulos e quais tem valores padrão. Foi usado para confirmar que todas as colunas foram criadas corretamente, inclusive a usuario_id adicionada pelo ALTER TABLE.

8.3 Diagrama Entidade-Relacionamento (DER)

O Diagrama Entidade-Relacionamento, chamado de DER, e uma representação visual das tabelas de um banco de dados e de como elas se conectam entre si. Ele ajuda a entender a estrutura do banco de um jeito mais intuitivo do que ler o código SQL diretamente. A seguir, o DER do GeoRisk Monitor e apresentado por meio de blocos que representam cada tabela e seus campos.

```
+---------------------------+
|         USUARIOS          |
+---------------------------+
| PK  id       INT  AI      |
|     nome     VARCHAR(50)  |
| UQ  email    VARCHAR(100) |
|     telefone VARCHAR(20)  |
|     senha    VARCHAR(20)  |
+---------------------------+
          | 1
          | (um usuario possui muitos alertas)
          | N
          v
+-------------------------------+
|            ALERTAS            |
+-------------------------------+
| PK  id         INT  AI        |
|     tipo       VARCHAR(50)    |
|     comentario TEXT           |
|     latitude   DOUBLE         |
|     longitude  DOUBLE         |
|     data       TIMESTAMP      |
| FK  usuario_id INT  (null ok) |
+-------------------------------+

+------------------------------+
|         PONTO_CRITICO        |
|   (entidade independente)    |
+------------------------------+
| PK  id                INT  AI |
|     nome              VARCHAR |
|     latitude          DOUBLE  |
|     longitude         DOUBLE  |
|     risco_alagamento BIT(1)   |
+------------------------------+
```

Legenda:

```
PK  = Primary Key (Chave Primaria) — identificador unico do registro
FK  = Foreign Key (Chave Estrangeira) — campo que aponta para outra
tabela
UQ  = Unique — valor que nao pode se repetir na tabela
AI  = Auto Increment — gerado automaticamente pelo banco
1:N = Um usuario pode ter muitos alertas (relacionamento um para
muitos)
```

O diagrama mostra três entidades. A tabela USUARIOS fica no topo porquê e a entidade central do sistema — e a partir dela que os alertas são criados. A seta apontando para baixo representa o relacionamento: um usuario pode ter zero ou vários alertas vinculados a ele, mas cada alerta pertence a no máximo um usuario. Isso e o que chamamos de relacionamento um para muitos, indicado pelo 1 próximo a USUARIOS e N próximo a ALERTAS.

A chave estrangeira usuario_id dentro da tabela ALERTAS e o campo que faz essa conexão. Ela guarda o número de id do usuario que criou o alerta. Quando esse campo está vazio — o que o banco indica como NULL — significa que o alerta foi gerado automaticamente pelo sistema, sem autoria humana.

A tabela PONTO_CRITICO existe de forma independente nesta versão do sistema. Ela nao tem relacionamento direto com USUARIOS ou ALERTAS, mas ja está estruturada para guardar locais com histórico de alagamento que poderão ser exibidos no mapa em versões futuras.

8.4 ERP — Entidade, Relacionamento e Persistência

O termo ERP neste contexto se refere a como as entidades do sistema são representadas no código Java e como elas se relacionam com o banco de dados por meio da camada de persistência do Spring Boot. E a ponte entre o mundo do Java e o mundo do SQL.

Cada tabela no banco de dados corresponde a uma classe Java chamada de Entity. A classe Usuario representa a tabela usuarios. A classe Alerta representa a tabela alertas. A classe Ponto Critico representa a tabela ponto

crítico. Essas classes são anotadas com @Entity, o que diz ao Hibernate que elas devem ser tratadas como tabelas no banco.

O relacionamento entre USUARIOS e ALERTAS que vimos no DER e representado no código Java pela anotação @ManyToOne dentro da classe Alerta. Ela diz: muitos alertas podem estar associados a um mesmo usuario. O campo correspondente no banco e o usuario_id, mapeado pela anotação @JoinColumn(name = "usuario_id").

A camada de repositório e responsável por executar as operações no banco. A interface AlertaRepository herda de JpaRepository e oferece automaticamente os métodos para salvar, buscar, listar e excluir alertas sem que o desenvolvedor precise escrever uma linha de SQL. O mesmo vale para UsuarioRepository em relação a tabela de usuarios. O Spring Data JPA gera o SQL necessário por baixo dos panos, com base nos nomes dos métodos definidos na interface.

Quando o servidor e iniciado, o Hibernate compara as classes Entity com o Chema existente no banco e aplica as mudanças necessárias. Se uma coluna nova foi adicionada na classe, mas ainda nao existe na tabela, o Hibernate a cria. Isso e o que aconteceu com a coluna usuario_id: ela foi adicionada primeiro via ALTER TABLE no MySQL Workbench e depois mapeada no código Java, garantindo que os dois lados fiquem sincronizados.

8.5 Configuração da Conexão com o Banco

Todas as configurações de conexão com o banco de dados ficam no arquivo application.properties, dentro da pasta src/main/resources do projeto. Esse arquivo e lido pelo Spring Boot quando o servidor sobe e define como a aplicacao vai se comunicar com o MySQL.

```
spring.datasource.url=jdbc:mysql://localhost:3306/climas_rotas
  ?useSSL=false&allowPublicKeyRetrieval=true
  &serverTimezone=America/Sao_Paulo
spring.datasource.username=root
```

```
spring.datasource.password=040622
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8D
ialect
jwt.secret=georisk-chave-secreta-super-segura-2024-nao-compartilhe
jwt.expiration=86400000
server.port=8080
```

O endereço de conexão aponta para o banco climas_rotas rodando localmente na porta 3306, que e a porta padrão do MySQL. O parâmetro useSSL=false desativa a exigência de conexão criptografada, o que é aceitável em ambiente local. O allowPublicKeyRetrieval=true e necessário por conta do método de autenticação do MySQL 8. O serverTimezone=America/Sao_Paulo garante que os horários gravados no banco fiquem no fuso correto — sem isso, os registros apareceriam com 3 horas de diferença.

O ddl-auto=update faz o Hibernate criar ou atualizar as tabelas automaticamente ao iniciar. O show-sul=true exibe no terminal do servidor cada comando SQL executado, o que e muito útil durante o desenvolvimento para acompanhar o que está acontecendo. O jwt.expiration=86400000 define que o token de autenticação expira em 24 horas — esse número representa 86 milhões e 400 mil milissegundos, que é exatamente um dia.

```
CREATE DATABASE climas_rotas;

USE climas_rotas;


CREATE TABLE usuarios (

    id INT AUTO_INCREMENT PRIMARY KEY,

    nome VARCHAR(50),

    email VARCHAR(100) UNIQUE,
```

```
    telefone VARCHAR(20),
    senha VARCHAR(20)
);
SHOW DATABASES;
SHOW TABLES;

INSERT INTO usuarios (nome, email, telefone, senha)
VALUES ('Teste', 'teste@email.com', '11999999999', '123456');

SELECT * FROM usuarios;

CREATE TABLE alertas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tipo VARCHAR(50),
    comentario TEXT,
    latitude DOUBLE,
    longitude DOUBLE,
    data TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
ALTER TABLE alertas ADD COLUMN usuario_id INT;
DESCRIBE alertas;
```

9. Fluxos Principais do Sistema

9.1 Fluxo de Cadastro

O usuario acessa a página principal e clica na aba cadastrar. Preenche nome, sobrenome, e-mail, telefone, senha e confirmação de senha. O JavaScript verifica os campos antes de enviar: se algum estiver vazio, se o e-mail nao tiver o formato correto, se a senha tiver menos de 6 caracteres ou se as duas senhas nao coincidirem, uma mensagem de erro aparece e o envio e bloqueado.

Se tudo estiver correto, o frontend envia os dados para o Firebase Authentication, que cria a conta e devolve um objeto de usuario autenticado. Em seguida, o frontend grava o perfil do usuario na coleção usuarios do Firestore. Com isso concluído, o usuario e redirecionado automaticamente para a tela do mapa, onde verá a mensagem de boas-vindas com o próprio nome.

9.2 Fluxo de Login

O usuario acessa a página principal e clica na aba entrar. Preenche e-mail e senha. O frontend envia essas informações para o Firebase Authentication, que verifica se o e-mail existe e se a senha corresponde ao hash armazenado. Se a autenticação falhar, uma mensagem de erro especifica e exibida. Se for bem-sucedida, o usuario e redirecionado para o mapa.

Existe também a opção esqueceu a senha. Quando o usuario clica, o sistema pede o e-mail e envia um link de redefinição através do Firebase. O usuario recebe o e-mail, clica no link e consegue criar uma nova senha sem precisar falar com ninguém da equipe.

9.3 Fluxo de Registro de Alerta

Com o mapa aberto, o usuario vê a área monitorada delimitada por um círculo. Ao clicar em qualquer ponto dentro desse círculo, uma janela se abre pedindo

o tipo de alerta e um comentario opcional. Ao confirmar, o frontend envia uma requisição para o servidor Java, incluindo o token JWT no cabeçalho para provar que o usuario está autenticado.

O servidor recebe a requisição, o JwtFilter valida o token e extrai o ID do usuario. O AlertaController salva o alerta no banco de dados, associado ao usuario que o criou. O Firestore e notificado e distribui o novo alerta para todos os outros clientes conectados, que veem o marcador aparecer no mapa em tempo real, sem precisar atualizar a página.

Se o usuario clicar fora do círculo monitorado, o sistema exibe um aviso no mapa e nao abre o formulário de registro. Isso evita que alertas sejam cadastrados em locais que estão fora da área de cobertura do sistema.

9.4 Fluxo de Atualização Automática

Um contador regressivo no cabeçalho da tela mostra quanto tempo falta para a próxima atualização. A cada 5 minutos, o sistema chama automaticamente a função que busca os dados climáticos da API Open-Meteo, recalcula o índice de risco e atualiza o painel lateral com as novas informações. Se as condições climáticas atingirem os limiares de alerta, novos marcadores automáticos são adicionados ao mapa.

10. Riscos e Limitações

10.1 Riscos Conhecidos

Indisponibilidade da API Open-Meteo. Se a API Open-Meteo ficar fora do ar, o sistema nao consegue atualizar os dados climáticos nem calcular o índice de risco. Nesses casos, o painel lateral exibe uma mensagem de erro, mas o mapa continua funcionando com os alertas existentes.

Cota do Firebase gratuito. O projeto usa o plano gratuito do Firebase, que tem limites de leituras e escritas por dia. Se o sistema tiver muitos usuarios simultâneos em um dia de emergência climática, esses limites podem ser atingidos e a sincronização em tempo real pode parar.

Credenciais no código-fonte. A chave secreta do JWT e a senha do banco de dados estão no arquivo application.properties. Se esse arquivo for enviado para um repositório público, essas informações ficam expostas. Em produção, elas devem ser configuradas como variáveis de ambiente no servidor, nunca diretamente no código.

Recusa de permissão de localização. Se o usuario nao der permissão para o navegador acessar a localização do dispositivo, o mapa centraliza no ponto de referência do Tatuapé e os dados climáticos são buscados para esse ponto fixo, nao para a localização real do usuario.

CORS aberto em desenvolvimento. O servidor está configurado para aceitar requisições de qualquer origem. Em produção, isso deve ser restringido apenas ao domínio do sistema.

10.2 Limitações da Versão Atual

O sistema de autenticação usa dois mecanismos independentes: o Firebase Authentication no frontend e o JWT gerado pelo Spring Boot no backend. Eles nao estão integrados entre si. Isso significa que o frontend pode estar autenticado pelo Firebase enquanto o backend usa um token JWT próprio —

duas sessões separadas que precisam ser mantidas sincronizadas pelo código da aplicacao.

Os alertas automáticos são criados e exibidos apenas no frontend, no momento em que os dados climáticos são processados. Eles nao são salvos no banco de dados MySQL. Isso significa que, se o usuario fechar e reabrir a página, os alertas automáticos da sessão anterior somem. Apenas os alertas manuais registrados pelos usuarios ficam persistidos no banco.

A área monitorada está definida como uma constante no código do frontend: 1.700 metros de raio ao redor do Tatuapé. Para alterar essa área ou adicionar novas regiões monitoradas, e necessário editar o código-fonte e reimplantar o sistema.

11. Dependências Externas

O GeoRisk Monitor depende de alguns serviços e bibliotecas externas para funcionar corretamente. Qualquer problema em um desses serviços pode afetar o funcionamento do sistema.

API Open-Meteo. Fornece todos os dados climáticos em tempo real. E gratuita, nao exige cadastro e tem disponibilidade histórica acima de 99,9%. Os dados incluem temperatura, vento, precipitação, umidade e previsão de 7 dias.

Firebase Authentication. Gerencia o cadastro e login dos usuarios no frontend. Faz parte dos serviços do Google Cloud.

Firebase Firestore. Banco de dados em nuvem usado para sincronizar os alertas em tempo real entre todos os usuarios conectados. Também faz parte do Google Cloud.

Leaflet.js. Biblioteca JavaScript de código aberto usada para renderizar o mapa interativo. E carregada via CDN e nao precisa ser instalada.

CartoDB Dark Matter. Fornece o visual escuro do mapa base, em camadas de imagem chamadas tiles. E um serviço gratuito para projetos nao comerciais.

Fonte Avexo-me e Google Fontes. Bibliotecas usadas para ícones e tipografia da interface. São carregadas via CDN.

Segue QRcode para repositório no Git

12. Referencias

AGAFONKIN, V. Leaflet.js — an open-source JavaScript library for mobile-friendly interactive maps. Disponivel em: https://leafletjs.com. Acesso em: abr. 2025.

AMBLER, S. W.; SADALAGE, P. J. Refactoring Databases: Evolutionary Database Design. Boston: Addison-Wesley, 2006.

BRASIL. Lei n. 13.709, de 14 de agosto de 2018. Lei Geral de Protecao de Dados Pessoais (LGPD). Diario Oficial da Uniao, Brasilia, 2018.

CENTRO DE GERENCIAMENTO DE EMERGENCIAS CLIMATICAS - CGE-SP. Relatorio de ocorrencias de alagamento na cidade de Sao Paulo. Sao Paulo: Prefeitura Municipal de Sao Paulo, 2023.

FIELDING, R. T. Architectural Styles and the Design of Network-based Software Architectures. Tese (Doutorado) — University of California, Irvine, 2000.

INSTITUTO DE PESQUISAS TECNOLOGICAS - IPT. Orientacoes para o gerenciamento de riscos: identificacao, analise, avaliacao e monitoramento. Sao Paulo: IPT, 2021.

NASCIMENTO, F. A.; SILVA, R. M. da. Sistemas de suporte a decisao em tempo real para gestao de riscos urbanos: uma revisao sistematica. Revista Brasileira de Cartografia, v. 73, n. 2, p. 410-425, 2021.

OPEN-METEO. Open-Meteo Weather API Documentation. Disponivel em: https://open-meteo.com/en/docs. Acesso em: abr. 2025.

PINHO, A. S.; COSTA, L. F. da. Inclusao digital e acesso a servicos de informacao em municipios brasileiros. Ciencia da Informacao, v. 51, n. 1, p. 78-95, 2022.

PROVOS, N.; MAZIERES, D. A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference, FREENIX Track, 1999.

SPRING FRAMEWORK. Spring Boot Reference Documentation. Disponivel em: https://docs.spring.io/spring-boot/docs/current/reference/html. Acesso em: abr. 2025.

W3C. Geolocation API. W3C Recommendation. Disponivel em: https://www.w3.org/TR/geolocation. Acesso em: abr. 2025.

Fim do Documento — GeoRisk Monitor v3.0 | Sao Paulo, 2025